

Foundations of StyleGAN

[StyleGAN]

1.0 Content Block

The Style Generative Adversarial Network, or StyleGAN for short, is an extension to the GAN architecture introduced by Nvidia researchers in December 2018, and made source available in February 2019. StyleGAN depends on Nvidia's CUDA software, GPUs, and Google's TensorFlow, or Meta AI's PyTorch, which supersedes TensorFlow as the official implementation library in later StyleGAN versions. The second version of StyleGAN, called StyleGAN2, was published on February 5, 2020. It removes some of the characteristic artifacts and improves the image quality. Nvidia introduced StyleGAN3, described as an "alias-free" version, on June 23, 2021, and made source available on October 12, 2021.

2.0 Content Block

StyleGAN3 StyleGAN3 improves upon StyleGAN2 by solving the "texture sticking" problem, which can be seen in the official videos. They analyzed the problem by the Nyquist–Shannon sampling theorem, and argued that the layers in the generator learned to exploit the high-frequency signal in the pixels they operate upon. To solve this, they proposed imposing strict lowpass filters between each generator's layers, so that the generator is forced to operate on the pixels in a way faithful to the continuous signals they represent, rather than operate on them as merely discrete signals. They further imposed rotational and translational invariance by using more signal filters. The resulting StyleGAN-3 is able to generate images that rotate and translate smoothly, and without texture sticking.

3.0 Content Block

Progressive GAN Progressive GAN is a method for training GAN for large-scale image generation stably, by growing a GAN generator from small to large scale in a pyramidal fashion. Like SinGAN, it decomposes the generator as $G = G_1 \circ G_2 \circ \dots \circ G_N$, and the discriminator as $D = D_N \circ D_{N-1} \circ \dots \circ D_1$. During training, at first only G_N, D_N are used in a GAN game to generate 4x4 images. Then G_{N-1}, D_{N-1} are added to reach the second stage of GAN game, to generate 8x8 images, and so on, until we reach a GAN game to generate 1024x1024 images. To avoid discontinuity between stages of the GAN game, each new layer is "blended in" (Figure 2 of the paper). For example, this is how the second stage GAN game starts:

4.0 Content Block

Illicit use In December 2019, Facebook took down a network of accounts with false identities, and mentioned that some of them had used profile pictures created with machine learning techniques.

5.0 Content Block

History A direct predecessor of the StyleGAN series is the Progressive GAN, published in 2017. In December 2018, Nvidia researchers distributed a preprint with accompanying software introducing StyleGAN, a GAN for producing an unlimited number of (often convincing) portraits of fake human faces. StyleGAN was able to run on Nvidia's commodity GPU processors. In February 2019, Uber engineer Phillip Wang used the software to create the website This Person Does Not Exist, which displayed a new face on each web page reload. Wang himself has expressed amazement, given that humans are evolved to specifically understand human faces, that nevertheless StyleGAN can competitively "pick apart all the relevant features (of human faces) and recompose them in a way that's coherent." In September 2019, a website called Generated Photos published 100,000 images as a collection of stock photos. The collection

was made using a private dataset shot in a controlled environment with similar light and angles. Similarly, two faculty at the University of Washington's Information School used StyleGAN to create Which Face is Real?, which challenged visitors to differentiate between a fake and a real face side by side. The faculty stated the intention was to "educate the public" about the existence of this technology so they could be wary of it, "just like eventually most people were made aware that you can Photoshop an image". The second version of StyleGAN, called StyleGAN2, was published on February 5, 2020. It removes some of the characteristic artifacts and improves the image quality. In 2021, a third version was released, improving consistency between fine and coarse details in the generator. Dubbed "alias-free", this version was implemented with PyTorch.

6.0 Content Block

Just before, the GAN game consists of the pair G_N, D_N generating and discriminating 4×4 images. Just after, the GAN game consists of the pair $((1 - \alpha) + \alpha \cdot G_{N-1}) \circ u \circ G_N, D_N \circ d \circ ((1 - \alpha) + \alpha \cdot D_{N-1})$ generating and discriminating 8×8 images. Here, the functions u, d are image up- and down-sampling functions, and α is a blend-in factor (much like an alpha in image compositing) that smoothly glides from 0 to 1.

7.0 Content Block

StyleGAN is designed as a combination of Progressive GAN with neural style transfer. The key architectural choice of StyleGAN-1 is a progressive growth mechanism, similar to Progressive GAN. Each generated image starts as a constant $4 \times 4 \times 512$ array, and repeatedly passed through style blocks. Each style block applies a "style latent vector" via affine transform ("adaptive instance normalization"), similar to how neural style transfer uses Gramian matrix. It then adds noise, and normalize (subtract the mean, then divide by the variance). At training time, usually only one style latent vector is used per image generated, but sometimes two ("mixing regularization") in order to encourage each style block to independently perform its stylization without expecting help from other style blocks (since they might receive an entirely different style latent vector). After training, multiple style latent vectors can be fed into each style block. Those fed to the lower layers control the large-scale styles, and those fed to the higher layers control the fine-detail styles. Style-mixing between two images x, x' can be performed as well. First, run a gradient descent to find z, z' such that $G(z) \approx x, G(z') \approx x'$. This is called "projecting an image back to style latent space". Then, z can be fed to the lower style blocks, and z' to the higher style blocks, to generate a composite image that has the large-scale style of x , and the fine-detail style of x' . Multiple images can also be composed this way.